Retrospectiva Ciclo 1:

1.¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Miniciclo 1:

- Modificación del tamaño de creación de canvas
- Modificar el método getCanvas
- Modificación del método constructor de circle y rectangle
- setPosition(int x, int y) en Rectangle y en Circle
- Creación de HashMap con estándar CSS

Justificación: Se debe modificar el paquete canvas para poder hacer que el canvas tenga un tamaño más grande y que por ende sea más visible la máquina, también unos métodos que permitan modificar la posición de objetos Rectangle y Circle de manera más simple y no por movimientos continuos, también se debe modificar el método constructor de Rectangle y Circle porque los constructores originales asignaban colores automáticamente, lo cual no sirve como requisito de funcionalidad, también se debe crear un Hashmap para poder cumplir con el requisito de colores de CSS.

Miniciclo 2:

- Symbol(String color, int x, int y)
- getColor()
- makeVisible()
- makeInvisible()
- isSameSymbol(Symbol sym2)
- setPosition(int x, int y)
- isVisible()

Justificación: En este miniciclo se construye toda la clase Symbol ya que son pocos los métodos que hay que definir y son poco complejos.

Miniciclo 3:

- Wheel(int x, int y)
- getCurrentSymbol()
- makeVisible()
- makeInvisible()
- updateSymbol()

Justificación: Se seleccionaron estos métodos para este miniciclo ya que algunos de estos tienen dependencias entre sí para poder funcionar en su totalidad.

Miniciclo 4:

- addSymbol(int pos, String color)
- removeSymbol(String color)
- placeSymbol(String color)
- spin()

Justificación: Se seleccionaron estos métodos para este miniciclo ya que eran los que más trabajaban con la clase Symbol y se podían plantear de forma similar sin perder el hilo en tanto al requisito de que las posiciones se enumeran a partir de 1.

Miniciclo 5:

- SlotMachine()
- addWheel(pos)
- delWheel(pos)
- addSymbol(pos, color)
- delSymbol(symbol)

Justificación: Sin ruedas ni símbolos no se puede ver ni probar nada, tambien los agrupe por que comparten el requisito de que las posiciones se enumeran a partir de 1 y de propagar los cambios a las otras clases.
Miniciclo 6:

- placeSymbol(wheel, symbol)
- spin()
- spin(wheel)
- isJackpot()

Justificación: Son los que definen lo que muestra cada rueda y los que verifican si hay jackpot, así que estos van juntos por que sin ellos jackpot no tendría nada que evaluar.
Miniciclo 7:

- Symbols()
- configuration()
- distinctSymbols()
- makeVisible()
- makeInvisible()
- exit()
- ok()
- showError(message)

Justificación: Se seleccionaron estos métodos para este miniciclo ya que son los que leen o muestran el estado de la máquina con avisos visuales al usuario y no generan modificaciones lógicas sobre ella.

2.¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?
Ya se terminaron todos los miniciclos planteados
3.¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)
17 Horas / Samuel
10 Horas / Sofía
4.¿Cuál considera fue el mayor logro? ¿Por qué?
El mayor logro fue conseguir la mayoría de funcionalidades propuestas en el ciclo con pocos bugs de interfaz que no pudimos solucionar.
5.¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
El mayor problema técnico fue de dibujo de canvas ya que al terminar el código, la lógica de dibujado funcionaba, pero como en el canvas los colores se dibujan encima de otras al hacer cambios sobre la máquina como al actualizar el estado de jackpot a true, se dibujaba la máquina encima de todo lo otro o casos en los que las ruedas se dibujan encima de los símbolos, asi que para resolver eso tuvimos que volver a revisar el código desde wheels para realizar cambios en el orden de algunas líneas y hacer prueba y error hasta que se dibujó todo en el orden necesario.

6.¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
Lo que hicimos bien como equipo fue distribuir el trabajo cuando nos dimos cuenta que no alcanzamos si hacíamos todo al mismo tiempo, para mejorar los resultados nos comprometemos a dedicar más tiempo al proyecto y organizarnos mejor.
7.Considerando las prácticas XP incluidas en los laboratorios. ¿Cuál fue la más útil? ¿por qué?

El trabajo por módulos, ya que se hizo progresivamente de lo más complicado a lo mas difícil, comenzando por modificar los métodos canvas, después crear la clase symbol, la clase wheel y por último la clase SlotMachine.
8.¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.
Se utilizó Claude – Anthropic. (2026). *Claude* (Sonnet 5) [Modelo de lenguaje de gran escala]. https://claude.ai – como asistente para estructurar el código de mapeo de colores CSS en la clase Canvas, y como referencia de consulta el estándar CSS Color Module Level 4 del W3C para obtener los 147 nombres de colores estándar y sus valores hexadecimales correspondientes – W3C. (2022). *CSS Color Module Level 4*. World Wide Web Consortium. https://www.w3.org/TR/css-color-4/ – .

Retrospectiva Ciclo 2:

1.¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Miniciclo 1:
swap(wheel1, wheel2)
Justificación: Se tomó este método como primer miniclip ya que es la funcionalidad que consideramos más fácil de implementar entre las 4 nuevas.

Miniciclo 2:
lock(wheel)
unlock(wheel)
getLocked()
setLocked() en Wheel
Justificación: Se agruparon porque estos son necesarios para manipular completamente el estado locked, además de esto se debieron agregas las validaciones de bloqueo que debian agregarse a los métodos ya existentes de ciclo 1, tomando decisiones de diseño de cómo se iba a comportar cada uno de estos con respecto al nuevo estado posible de las ruedas, los métodos modificados ya existentes fueron (placeSymbol(String : color), spin(), spin(wheel)).

Miniciclo 3:
spin(wheel, steps) en SlotMachine
spin(steps) en Wheel
Justificación: Se trabajaron juntos porque funcionan tipo cascada ya que en SlotMachine se hacen las validaciones y en Wheel se ejecuta toda la lógica para hacer que el índice de Wheel se desplace de forma circular sobre los símbolos según la cantidad dada de pasos.

Miniciclo 4:
spin(setSymbols[]) en SlotMachine
Justificación: Se dejó de últimas porque es la funcionalidad más completa y compleja, ya que tiene bastantes validaciones, entre ellas verificar que los colores pertenezcan a la lista de masterSymbols, que el tamaño de símbolos no sea mayor a la cantidad de símbolos en masterSymbols etc.

2.¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?
Ya se terminaron todos los miniciclos planteados.

3.¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)
13 Horas / Samuel
4 Horas / Sofía

4.¿Cuál considera fue el mayor logro? ¿Por qué?
El mayor logro fue identificar y corregir bugs del ciclo 1 que no habíamos podido identificar en ese entonces por la falta de conocimiento sobre el manejo de pruebas unitarias.

5.¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
Gracias al manejo de las pruebas unitarias en este ciclo sobre todo en la función lock que decidimos ejecutar pruebas que incluyen asserts sobre coincidencia de colores, identificamos errores en Wheel : addSymbol donde no se estaba ajustando el índice al insertar un nuevo símbolo lo que podría generar un cambio de símbolo dentro de la rueda sin que el usuario lo hubiera pedido y generaba errores de assert.
La solución fue incrementar index cuando la posición de inserción es menor o igual al índice actual protegido con la variable empty para no afectar el caso de la primera inserción de una rueda vacía en donde el índice debe mantenerse en 0.

identificamos errores en SlotMachine : isJackpot donde no había protección contra las ruedas sin símbolos, ya que dentro de este se llamaba getCurrentSymbol() en cada rueda sin verificar que tuviera símbolos, lo que podía producir IndexOutOfBoundsException.
La solución fue agregar un condicional que retornará en caso de que la rueda inicial o alguna rueda dentro del ciclo no tenga símbolos.

identificamos un error en SlotMachine : updateJackpotStatus() que forzaba la visibilidad de la rueda incondicionalmente, lo cual descubrimos porque en el enunciado se pedía la condición de que las pruebas debían ser ejecutadas en modo Invisible y al ejecutar cierta prueba se hacia todo visible.
La solución fue agregar un condicional que hiciera visible para visibilizar los cambios en caso de que la máquina ya fuera visible.

También hubo algunos problemas en el orden de verificación de condiciones dentro de placeSymbol() y spin(wheel: int), ya que se estaba verificando que no hubieran ruedas bloqueadas antes de verificar si siquiera existían ruedas, la solución fue intercambiar el orden.

6.¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
Lo que hicimos bien como equipo fue distribuir el trabajo cuando nos dimos cuenta que no alcanzamos si hacíamos todo al mismo tiempo, para mejorar los resultados nos comprometemos a dedicar más tiempo al proyecto y organizarnos mejor.

7.Considerando las prácticas XP incluidas en los laboratorios. ¿Cuál fue la más útil? ¿por qué?
Lo más útil fueron las pruebas unitarias, ya que permiten hacer seguimiento al codigo y encontrar bugs fácilmente.

8.¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.
Se utilizó Claude – Anthropic. (2026). *Claude* (Sonnet 5) [Modelo de lenguaje de gran escala]. https://claude.ai – como asistente para estructurar el código de mapeo de colores CSS en la clase Canvas, y como referencia de consulta el estándar CSS Color Module Level 4 del W3C para obtener los 147 nombres de colores estándar y sus valores hexadecimales correspondientes – W3C. (2022). *CSS Color Module Level 4*. World Wide Web Consortium. https://www.w3.org/TR/css-color-4/ – .
Se consultó Junit https://docs.junit.org/current/user-guide/ como referencia para el uso de @BeforeEach, @AfterEach y @Test en las pruebas unitarias.

Retrospectiva Ciclo 3:

1.¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.
Miniciclo 1
Creación de Shape como clase padre
Modificación de Circle, Rectangle y Triangle para heredar de Shape
Justificación: Se debía aplicar herencia entre las clases del paquete shapes ya que había mucho código repetido y solo unos pocos métodos y atributos que varían entre clases.

Miniciclo 2:
SlotMachine(int n)
spin(int steps) en Wheel rotación inversa
spin(int wheel, int steps)
getWheel(int wheel)
getSymbol(int pos)
Justificación: Se debía crear un método constructor sobrecargado que pudiera utilizar la clase SlotMachineContest para poder simular SlotMachine, se hicieron cambios en los spin steps ya que solo caminaban en un sentido y para el problema deben de caminar en ambos sentidos, también se modificaron los métodos getWheel y getSymbol ya que funcionaban asi no existieran ni ruedas ni símbolos.

Miniciclo 3:
simulate(int n)
Justificación: Ya que se debe crear un algoritmo similar para los métodos de SlotMachineContest decidí primero realizar el de simulate ya que es más sencillo porque no debe retornar ningún arreglo y por que ya me acostumbre a usar POO.

Miniciclo 4:
solve(n)
distinctSymbols(int[] indexes, int n)
spin(int wheel, int steps, int n, int[] indexes, ArrayList<int[]> movesList)
Justificación: Para el método solve se requería k que es la cantidad de símbolos distintos en SlotMachine y spin para poder rotas las ruedas, por eso decidí dejarlas en el mismo ciclo.
2.¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?
Ya se terminaron todos los miniciclos planteados.
3.¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)
12 Horas / Samuel
4.¿Cuál considera fue el mayor logro? ¿Por qué?
El mayor logro fue plantear la estrategia para resolver el problema, ya que al principio intente resolverlo sin plantear una estrategia y codificando sin mente, pero llegaba un punto en el que solo habían dos símbolos distintos en toda la máquina, pero el algoritmo no lograba distinguir qué símbolo tiene cada rueda y quedaba en un ciclo infinito, así que tuve que diseñar la solución de una forma diferente.
5.¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
Recordar la solución modular que se utilizaba en materias anteriores.
Para resolverlo revise el código de python de materias anteriores y lo aplique de forma similar a java hasta que funciono.
6.¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
Mi compañera de equipo canceló la materia.
Debo organizar mejor mi tiempo.
7.Considerando las prácticas XP incluidas en los laboratorios. ¿Cuál fue la más útil? ¿por qué?
Refactorizacion ya que gracias a esta práctica introduje la clase Shape, modificar la rotación steps y agregar condicionales a getSymbol y getWheel sin romper nada.
8.¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.
Java Platform, Standard Edition Documentation.
https://docs.oracle.com/javase/9/docs/api/java/util/Random.html
Utilizada para generar números al azar en un rango definido.
https://docs.oracle.com/javase/7/docs/api/java/util/ArrayList.html
Para el método toArray para convertir el ArrayList con los pasos para lograr el jackpot en un arreglo int[][] sin complicarse innecesariamente.
https://docs.oracle.com/javase/8/docs/api/java/lang/IllegalArgumentException.html
utilizada para evitar que en getWheel y getSymbol se retorne null y pueda causar errores.
https://docs.oracle.com/javase/tutorial/essential/concurrency/sleep.html
Para que la rotación de las ruedas sea un poco más lenta y visible.
https://docs.oracle.com/javase/tutorial/essential/exceptions/catch.html
utilizada para capturar la InterruptedException que puede lanzar Thread.sleep.

Retrospectiva Ciclo 4:

1.¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Miniciclo 1:

- Wheel(int x, int y, String color) en Wheel
- setLeftNeighbor(Wheel leftNeighbor) en Wheel
- canLock(), canSwap() y canDelete() en Wheel
- Lefty(int x, int y)
- setLeftNeighbor(Wheel left) en Lefty
- spin() y spin(int steps) en Lefty
- copyLeft() en Lefty
- Rebel(int x, int y)
- canLock(), canSwap() y canDelete() en Rebel
- addWheel(String type, int pos)
- recalculateIndexes(int index, char op, char obj)
- recalculatePositions()
- lock(int wheel), swap(int wheel1, int wheel2) y delWheel(int pos)
- delSymbol(String symbol), spin(int wheel) y getWheel(int wheel)

Justificación: Se agruparon las ruedas Lefty y Rebel porque las dos parten de la clase Wheel. A Wheel se le agregaron los métodos setLeftNeighbor, canLock, canSwap y canDelete con el comportamiento de una rueda normal, para que las subclases solo sobrescriban lo que cambia y SlotMachine no tenga que preguntar de qué tipo es cada rueda. También se refactorizo recalculateIndexes para que retorne el índice ajustado, ya que todos los métodos que reciben posiciones repetían las mismas validaciones y se modificaron lock, swap y delWheel para que consulten a la rueda antes de actuar y se corrigieron delSymbol, spin(wheel) y getWheel.

Miniciclo 2:

- Symbol(String color, Shape figure) en Symbol
- getSize() en Symbol
- isHidden() en Symbol
- onStop() en Symbol
- copy() en Symbol
- Ephemeral(String color, int x, int y)
- getSize(), setPosition(int x, int y), onStop() y copy() en Ephemeral
- Shy(String color, int x, int y)
- getSize(), isHidden(), onStop(), makeVisible() y copy() en Shy
- addSymbol(int pos, Symbol sym) en Wheel
- updateSymbol() en Wheel
- spin() y spin(int steps) en Wheel
- addSymbol(String type, int pos, String color) en SlotMachine
- addWheel(int pos), addWheel(String type, int pos) y addSymbol(int pos, String color) en SlotMachine

Justificación: Se trabajaron juntos los dos tipos de símbolo porque comparten el mismo mecanismo: Symbol tiene el método onStop(), que la rueda llama cuando un giro termina en el símbolo, y cada tipo lo sobrescribe (Ephemeral se encoge y Shy alterna entre visible e invisible). Como cada rueda necesita su propio objeto con su propio estado, se agregó copy(), que crea un símbolo nuevo del mismo tipo y color que el símbolo maestro. Además, como los elementos de diferentes tipos deben poder distinguirse visualmente, la figura de Symbol pasó de Circle a Shape: Ephemeral es un triángulo y Shy es un cuadrado.

Miniciclo 3:

- SlotMachineException(String message)
- SlotMachineException(String message, Throwable cause)
- getWheel, getSymbol, lock, unlock, swap, delWheel y placeSymbol en SlotMachine
- spin(), spin(int wheel), spin(int wheel, int steps) y spin(String[] setSymbols) en SlotMachine
- delSymbol, addSymbol(int pos, String color) y addSymbol(String type, int pos, String color) en SlotMachine
- SlotMachine(int n)
- solve(int n) y simulate(int n) en SlotMachineContest

Justificación: Se dejó de último porque modifica casi todos los métodos de SlotMachine y también las pruebas. Se eliminaron showError y los JOptionPane y cada showError con su return pasó a ser un throw de SlotMachineException.

2.¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?

Se terminaron los miniciclos 1, 2 y 3.

3.¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

17 Horas / Samuel

4.¿Cuál considera fue el mayor logro? ¿Por qué?

El mayor logro fue conseguir que los tipos de rueda y de símbolo funcionaran con herencia sin que SlotMachine tuviera que preguntar de qué tipo era cada uno, ya que así agregar un tipo nuevo solo requiere crear una clase y una rama en addWheel o addSymbol.

5.¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fue diferenciar visualmente los símbolos Ephemeral y Shy. Al cambiar la figura de Symbol de Circle a Shape, el triángulo quedaba descentrado dentro de la rueda porque Triangle ubica su posición x en la punta y no en el centro, y el cuadrado se veía demasiado grande con el mismo tamaño del círculo.

Para resolverlo se renombró el diámetro a size en todo el proyecto, updateSymbol() de Wheel pasó a centrar con getSize(), Ephemeral sobrescribió setPosition para sumar size / 2 en x y el lado de Shy se redujo a 15 hasta que se vio bien.

6.¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Mi compañera de equipo canceló la materia.

Debo organizar mejor mi tiempo para no dejar pendientes como el uso de Class.forName.

7.Considerando las prácticas XP incluidas en los laboratorios. ¿Cuál fue la más útil? ¿por qué?

Refactorización, ya que gracias a esta práctica pude reescribir recalculateIndexes, agregar los tipos de rueda y de símbolo, y reemplazar los JOptionPane por excepciones sin cambiar la estructura de los métodos y con todas las pruebas pasando.

8. Referencias

Uso de ClassForName para evitar condicionales en los métodos addSymbol y addWheel que reciben tipos de símbolos y tipos de ruedas respectivamente

https://www.geeksforgeeks.org/java/class-forname-method-in-java-with-examples/